

50 Master Exam Questions & Answers

Grounded in *Architectures and Applications of Syntax Analysis* — Comprehensive Study Bank

Category 1: Compiler Architecture & Execution Models

Q1: What is the primary difference between a compiler and an interpreter regarding execution speed and error reporting?

A compiler translates the entire high-level program into target machine code ahead of time. The resulting machine code executes rapidly, but all compilation errors are reported at once before execution. An interpreter translates and executes the source code line by line at runtime, which results in slower execution but provides immediate statement-by-statement error diagnostics.

Q2: How does a Just-In-Time (JIT) compiler combine compilation and interpretation?

A JIT compiler initially interprets high-level code or bytecodes (e.g., Java bytecodes). When hot execution paths or frequently called methods are detected at runtime, the JIT compiler dynamically translates those bytecodes directly into native machine code to drastically speed up execution.

Q3: What are the two main phases of a compiler architecture and their responsibilities?

The analysis phase (frontend) breaks down source code into constituent tokens, builds parse trees, checks grammatical and semantic validity, and builds an intermediate representation (IR) along with the symbol table. The synthesis phase (backend) takes the IR and symbol table to synthesize optimized target machine instructions.

Q4: Why is intermediate code generation advantageous in retargetable compilers?

Using a machine-independent intermediate representation decouples language frontends from hardware backends. To support M source languages and N target hardware architectures, one only needs M frontends and N backends ($M + N$ modules) instead of $M \times N$ distinct full compilers.

Q5: What is a transpiler (source-to-source compiler) and what is an example from compiler history?

A transpiler translates source code written in one high-level language into another high-level language. An influential example is Cfront (1983), the original C++ compiler, which transpiled C++ code into C code for compilation by standard C compilers.

Q6: How does an AST interpreter differ from a bytecode interpreter in execution performance?

An AST interpreter executes programs by walking nodes of an Abstract Syntax Tree directly, causing high pointer-dereferencing and tree-traversal overhead. A bytecode interpreter executes flat, 1-byte opcode instructions in a tight loop, offering superior performance.

Q7: How does a compiler resolve static function calls vs. virtual function calls in object-oriented inheritance?

Static non-virtual calls are resolved at compile time using scope lookup in the symbol table and direct name mangling to emit direct assembly call instructions. Virtual calls are dispatched dynamically at runtime via an indirect pointer lookup in the object's Virtual Method Table (vtable).

Q8: Do access modifiers (public, protected, private) alter executable assembly code or vtable structures?

No. Access modifiers are enforced strictly at compile time during semantic analysis. If access rules are violated, compilation halts with an accessibility error. At runtime, machine code and vtable layouts remain identical regardless of visibility tags.

Category 2: Lexical Analysis & Input Buffering

Q9: What is the distinction between a token, a pattern, and a lexeme?

A token is an abstract symbol (and optional attribute value) returned to the parser. A pattern is the formal rule (often a regular expression) describing matching strings. A lexeme is the actual character sequence in the source code that matches a token's pattern.

Q10: Why is input buffering necessary in lexical analysis?

Lexical analyzers frequently need to look ahead one or more characters beyond a token (e.g., distinguishing 'x' from 'x1' or '=' from '=='). Specialized buffering reduces disk I/O system calls by reading large blocks of characters into memory at once.

Q11: How does a two-buffer scheme with sentinels optimize character scanning?

Two buffers of size N (e.g., 4096 bytes) are alternately reloaded. By placing a special sentinel character (EOF) at the end of each buffer, the scanner performs only one test per character in the common case instead of testing for both buffer boundaries and character values.

Q12: What recovery strategy does a lexical analyzer use when encountering unrecognized input?

The simplest strategy is 'panic mode' recovery, where the scanner deletes successive characters from the remaining input until it finds a well-formed token prefix.

Q13: How does the 'lexemeBegin' pointer interact with the 'forward' pointer during scanning?

The 'lexemeBegin' pointer marks the start of the current candidate token. The 'forward' pointer advances character by character until a pattern match is completed, at which point 'lexemeBegin' moves to the position immediately following the lexeme.

Q14: How are regular definitions used to describe complex programming language tokens?

Regular definitions assign names to regular subexpressions. For example, 'digit -> [0-9]' and 'digits -> digit+', allowing complex tokens like floating-point numbers 'digits (. digits)?' to be specified concisely.

Q15: Why must lexical analyzer generators (like Lex/Flex) prioritize keyword patterns over identifier patterns?

Keywords (e.g., 'if', 'while') match the general pattern for identifiers. Scanner generators test patterns in order of specification, ensuring keywords are matched first before falling back to generic identifier tokens.

Q16: What is the role of token attribute values passed from the scanner to later compiler phases?

While the token name guides parsing decisions, the attribute value (e.g., a symbol table pointer or numeric value) conveys the specific instance data required for semantic analysis and code generation.

Category 3: Context-Free Grammars & Derivations

Q17: What are the four formal components of a Context-Free Grammar (CFG)?

A CFG consists of: (1) Terminals (basic tokens), (2) Nonterminals (syntactic variables), (3) A Start Symbol, and (4) Productions (rules mapping a nonterminal to a string of terminals and nonterminals).

Q18: What is the difference between a leftmost derivation and a rightmost derivation?

In a leftmost derivation, the leftmost nonterminal in a sentential form is rewritten at every step. In a rightmost derivation, the rightmost nonterminal is rewritten at every step.

Q19: What defines an ambiguous grammar?

A grammar is ambiguous if it can produce more than one parse tree (or equivalently, more than one leftmost derivation or rightmost derivation) for the same terminal string.

Q20: Why must immediate left recursion be removed before constructing a top-down parser?

Top-down parsers expand nonterminals from left to right. Left-recursive rules of the form ' $A \rightarrow A\alpha$ ' cause a recursive-descent parser to enter an infinite recursion loop without consuming input.

Q21: How can operator precedence be enforced in an arithmetic expression grammar?

By introducing distinct nonterminals for each precedence level. Higher precedence operators are placed deeper in the grammar hierarchy (farther from the start symbol) so they bind tighter in the parse tree.

Q22: What is the 'dangling-else' ambiguity problem and how is it traditionally resolved?

In statements like 'if e_1 then if e_2 then s_1 else s_2 ', the 'else' can attach to either 'if'. It is resolved by giving explicit parser preference to shift 'else', associating each 'else' with the nearest unmatched 'then'.

Q23: What is a sentential form in a grammar derivation?

A sentential form is any string of terminals and/or nonterminals derivable from the grammar's start symbol during a derivation sequence.

Q24: What is the relationship between parse trees and leftmost derivations?

There is a strict one-to-one correspondence between every unique parse tree and its unique leftmost derivation (and unique rightmost derivation).

Category 4: Top-Down Parsing, FIRST/FOLLOW & LL(1)

Q25: What does LL(1) stand for in compiler terminology?

Left-to-right scanning of the input, Leftmost derivation construction, using 1 symbol of lookahead to make parsing decisions.

Q26: How is the FIRST(X) set defined for a grammar symbol X?

FIRST(X) is the set of terminals that can begin strings derived from X. If X can derive epsilon (the empty string), epsilon is also in FIRST(X).

Q27: How is the FOLLOW(A) set defined for a nonterminal A?

FOLLOW(A) is the set of terminals that can appear immediately to the right of nonterminal A in some sentential form derived from the start symbol.

Q28: What is the fundamental condition for a grammar to be LL(1)?

For any nonterminal A with distinct productions $A \rightarrow \alpha \mid \beta$: (1) FIRST(α) and FIRST(β) must be disjoint, and (2) If epsilon is in FIRST(α), FIRST(β) and FOLLOW(A) must be disjoint.

Q29: How does a predictive LL(1) parser use an explicit stack to guide parsing?

The stack holds grammar symbols with the start symbol at top. The parser matches top-of-stack terminals against input, or pops top nonterminal A and pushes body α of production $M[A, a]$ where 'a' is lookahead.

Q30: What are synchronizing tokens in panic-mode top-down error recovery?

Synchronizing tokens are terminals (typically from FOLLOW(A)) placed in error entries of the parsing table. When an error occurs, the parser skips input until a synchronizing token appears, then pops A to resume.

Q31: What is left factoring and why is it applied to grammars?

Left factoring rewrites productions sharing a common prefix (e.g., $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ into $A \rightarrow \alpha A'$ and $A' \rightarrow \beta_1 \mid \beta_2$) to defer choices until enough input is seen for LL(1) parsing.

Q32: Why cannot left-recursive grammars ever be LL(1)?

Left recursion causes overlapping FIRST sets between production alternatives (e.g., $\text{FIRST}(A \alpha)$ contains $\text{FIRST}(\beta)$), violating the LL(1) disjointness rule.

Category 5: Bottom-Up Parsing & LR Automata

Q33: What is the fundamental mechanism of shift-reduce bottom-up parsing?

A shift-reduce parser constructs a parse tree from leaves to root (a rightmost derivation in reverse) by shifting input symbols onto a stack and reducing matching substrings (handles) to nonterminals.

Q34: What is a 'handle' in a right-sentential form?

A handle is a substring matching the body of a production rule whose reduction represents one step in the reverse of a rightmost derivation.

Q35: Where does the handle always reside in a shift-reduce parser?

The handle always resides at the top of the stack, never embedded inside lower stack levels.

Q36: What is a shift/reduce conflict in shift-reduce parsing?

A configuration where the parser cannot determine whether to shift the next input symbol onto the stack or reduce the current stack top by a production rule.

Q37: What is a reduce/reduce conflict in shift-reduce parsing?

A configuration where the parser knows a handle is on top of the stack but cannot determine which of two or more valid production rules to apply in the reduction.

Q38: What is an LR(0) item?

An LR(0) item is a production rule with a dot at some position in its body (e.g., $A \rightarrow X \cdot Y Z$), indicating how much of the production body has been parsed.

Q39: What is the purpose of the CLOSURE function in LR parser construction?

$\text{CLOSURE}(I)$ expands an item set I by adding items $[B \rightarrow \cdot \gamma]$ whenever an item $[A \rightarrow \alpha \cdot B \beta]$ is in the closure, tracking all productions that might be parsed next.

Q40: How does the GOTO(I, X) function work in LR(0) automata?

$\text{GOTO}(I, X)$ computes the closure of all items in I where the dot appears immediately before grammar symbol X , representing a state transition under X .

Q41: Why is the augmented grammar production $S' \rightarrow S$ introduced in LR automata?

To indicate unambiguous acceptance: the parser accepts the input string if and only if it is about to reduce by the top-level rule $S' \rightarrow S$.

Category 6: Syntax-Directed Translation & Attribute Evaluation

Q42: What is the difference between a synthesized attribute and an inherited attribute?

A synthesized attribute at a node is computed solely from attributes of its children or itself. An inherited attribute at a node is computed from attributes of its parent, siblings, or itself.

Q43: What defines an S-attributed Syntax-Directed Definition (SDD)?

An SDD is S-attributed if every attribute in the grammar is synthesized. S-attributed SDDs can be evaluated during bottom-up parsing using a postorder traversal.

Q44: What defines an L-attributed Syntax-Directed Definition (SDD)?

An SDD is L-attributed if inherited attributes depend only on parent inherited attributes or attributes of siblings strictly to the left in the production body.

Q45: What role does a dependency graph play in SDD evaluation?

A dependency graph depicts attribute dependencies in a parse tree. A topological sort of the dependency graph determines a valid linear order for evaluating all attributes.

Q46: How do Syntax-Directed Translation Schemes (SDTs) differ from SDDs?

SDDs provide high-level specifications without specifying evaluation timing. SDTs embed explicit programmatic actions directly into grammar production bodies for execution during parsing.

Category 7: Symbol Table Organization & Scope Management

Q47: What are the two primary questions a symbol table must answer?

1. Given a declaration, is the name already declared in the current scope (multiply declared)? 2. Given a name use, which declaration corresponds to it under the most-closely-nested rule?

Q48: How does the 'List of Hashtables' symbol table implementation operate?

Maintains a list where each hashtable represents a visible scope level. Scope entry adds a table to the front $O(1)$; scope exit removes the front table $O(1)$. Variable lookup searches tables from front to back $O(\text{depth})$.

Q49: How does the 'Hashtable of Lists' symbol table implementation operate?

Maintains one central hashtable mapping names to stacks of declarations. Variable lookup is expected $O(1)$. Scope exit requires scanning entries to remove level-N declarations $O(N)$.

Q50: Why are function names inserted into the outermost scope table rather than the function's local table?

So that function names remain visible for lookup when calls are processed inside other functions in enclosing or sibling scopes.